



The City College
of New York

CSC 59867-E: Senior Project II

AI Agents for Decision Making in the Real World

By Saptarashmi Bandyopadhyay

Email: sbandyopadhyay@ccny.cuny.edu

Assistant Professor of Computer Science

City College of New York and Graduate Center at the City University of New York

September 14, 2026 CSC 59867 - E

Sim2Real Physical Alignment (Coordinate Registration)

When a robot arm transfers plates we need to map coordinates with very high (often sub-mm) precision through a transformation matrix $\mathbf{T}_B^S \in SE(3)$ (Recall: $SE(3)$ is the set of possible rigid-body motions in 3D space)

We refine the spatial alignment between the arm's base $\{B\}$ and the shelf port $\{S\}$ through the following optimization problem (aka the orthogonal Procrustes problem):

$$\mathbf{R}_B^S, \mathbf{t}_B^S = \arg \min_{\mathbf{R} \in SO(3), \mathbf{t} \in \mathbb{R}^3} \sum_{i=1}^N \|\mathbf{p}_{B,i} - (\mathbf{R}\mathbf{p}_{S,i} + \mathbf{t})\|_2^2$$

Evaluating the SVD of the cross-covar matrix $\mathbf{H} = \mathbf{U}\Sigma\mathbf{V}^T$ gives us rotation $\mathbf{R}_B^S = \mathbf{U}\mathbf{V}^T$.



Scalable Active Learning with Bayesian Optimization

Active learning combines every step of the scientific process into an iterative cycle to continuously improve results. We know from lectures last semester how important consistent learning is.

Evaluating physical samples serially wastes robotic system resources.

We use multi-point Expected Improvement (q -EI) to evaluate batches of size q simultaneously.

The batch acquisition function is optimized using Monte Carlo integration and the reparameterization trick over the Cholesky decomposition of the predictive covariance:

$$\hat{\alpha}_{q\text{-EI}}(\mathbf{X}) = \frac{1}{M} \sum_{m=1}^M \max \left(0, \max_{i=1 \dots q} y_i^{(m)}(\mathbf{X}) - y^* \right)$$



High-Throughput Orchestration

AI Agents require distributed processing to orchestrate massive datasets:



JAXMARL

I helped create JAXMARL, a Multi-agent training framework that hosts environments and algorithms directly on the GPU through operations such as `vmap`, allowing training to scale across massive clusters (e.g. EmpireAI)



Ray Workflows

Models physical lab steps as durable, isolated DAG-tors. If a liquid handler faults, Ray restores state without corrupting prior analytical data.



ChemOS / PyLabRobot

Autonomous laboratories like AI-BIOME need Hardware-agnostic orchestration that allows for effective simulation and rigorous automated testing of hardware control for laboratories.



Case Study: Peptide Self-Assembly & Reward Hacking

Target Discovery

The goal is to Discover short peptide sequences that autonomously assemble into functional, structured nanomaterials. We're doing this in AI-BIOME now.

This system that we're considering relied on optical density (OD) readings as their main method of testing pass / fail situations in peptide assembly

Specification Gaming

An RL agent bypassed the hard chemistry by exploiting physical parameters on the automation deck:

- Running the on-deck heater at **85°C**
- Creating rapid evaporation through rapid mixing, exploiting their control over the automation.

Artificial Signal

The reduced liquid volume concentrated the solute, artificially driving OD readings upward.

Zero actual self-assembly occurred.

This is a great real-world example of how rigorously agents need to be tested before they can be released in the physical world.



Code Example: Enforcing Kinematic Envelope

```
def execute_safe_transfer(protocol, pipette, volume, source_well, dest_well):
    MAX_WELL_CAPACITY = 200.0
    current_dest_vol = get_tracked_volume(dest_well) # Custom tracking func

    # Kinematic Envelope & Spill Assertion
    if current_dest_vol + volume > MAX_WELL_CAPACITY:
        protocol.pause("Safety Intercept: Destination well capacity exceeded.")
        return

    pipette.aspirate(volume, source_well)
    protocol.delay(seconds=2.0) # Viscous liquid class mitigation
    pipette.dispense(volume, dest_well)
```



Recall from previous classes: Avoiding Reward Hacking

To prevent evaporative deception and precipitate scattering spoofs, we must formulate composite obj functions that regularize against mass loss.

The physical objective is regularized using real-time gravimetric feedback (mass deficits) and dual-wavelength IR scattering ($\lambda = 850\text{nm}$):

$$\mathcal{J}_{\text{reg}}(x) = f(x) - \lambda_m \left| m_{\text{final}} - \sum m_i \right| - \lambda_{\text{scat}} \mathbb{I} \left(\frac{A_{850}}{A_{600}} > \tau_{\text{turbid}} \right)$$

The HAL automatically penalizes policy returns if mass conservation is violated.



Out of Distribution Action Masking

We can't always rely on agents (which operate in high-d latent spaces) to always give physical decisions that are in line with real-world safety requirements. We can formalize this in the following way:

If the epistemic posterior variance $\sigma^2(x)$ predicted by the Gaussian Process exceeds a safety threshold τ_{OOD} , the system intercepts the command.

The physical action space is constrained using confidence-bound polytopes:

$$\mathcal{X}_{\text{safe}} = \{x \in \mathcal{X} \mid \mu(x) - \beta\sigma(x) \geq c_{\text{safe}}, \sigma^2(x) \leq \tau_{\text{OOD}}\}$$

Optimizing Latency, Memory Inflation, and Network Bandwidth

—



Topics to Cover

Motivation: If you end up doing anything computational at scale, you will certainly need to be aware of the tradeoffs and bottlenecks in distributed computing. If your project involves using Empire AI compute, this will be very helpful for navigating cluster computation jobs.

- Communication Penalties (e.g. trade-offs) and modeling physical delay links
- Pipelining and packetization in order to minimize latency in communication

Next lecture, we'll go more into:

- Diameters, degree constraints, and cross-section limits for topological bounds
- Iterative methods and their tradeoffs in balancing concurrency and communication
- Optimizing memory usage through smart computation of matrix rows and columns
- **Example Problem:** How to derive optimal cluster size for distributed inner products



Computation vs. Communication

In parallel numerical architectures, total execution time T_{TOTAL} is governed by both arithmetic operations and data movement. **Communication Penalty (CP):**

$$CP = \frac{T_{\text{TOTAL}}}{T_{\text{COMP}}}$$

where T_{COMP} is the time required if all interprocessor communication were instantaneous. You can think of this as the “cost” it takes to relay info between processors.

If an algorithm requires transmitting a packet across a link for every few numerical operations, communication time inevitably dominates execution time ($CP \gg 1$).

System Goal: Maximize speedup $S_p = T^*/T_p$ by minimizing the communication penalty while scaling processor count p . We are mostly interested in very large values for p .



Link Latency Model

Communication links act as asynchronous packet pipes connecting processors.
We can formalize the packet delay D for a point-to-point (P2P) in the following way:

$$D = P + RL + Q$$

- P : Processing and propagation delay (time to inspect headers, format packets, and traverse the physical medium).
- R : Transmission time per bit ($1/R$ represents link bandwidth or channel capacity).
- L : Packet length in bits (including algorithmic payload and protocol overhead).
- Q : Queueing delay (waiting time spent in intermediate buffers due to link contention or scarce receive buffers).

In tightly coupled clusters, P is often small, whereas RL or contention-induced Q dominates. By contention, we mean competition for the same resources, analogues to a zero-sum game in Game Theory.



Message Pipelining & Packet Segmentation

Transmitting a monolithic message of size M over a multi-hop path of k links requires store-and-forward delay:

$$T_{\text{unpipelined}} = k \cdot nT$$

where the message is divided into n segments, each requiring transmission time T on a single link.

Pipelined Transmission: Segmenting the message into n discrete packets allows intermediate nodes to forward packets concurrently along the path:

$$T_{\text{pipelined}} = (n + k - 1)T$$

When packet segmentation overhead is negligible, pipelining reduces latency over k links by a factor approaching k

Cut-through Routing: Processors forward packets as soon as headers are decoded, without waiting for the trailing payload to arrive. This way, we can process multiple packets simultaneously.

Questions?

—